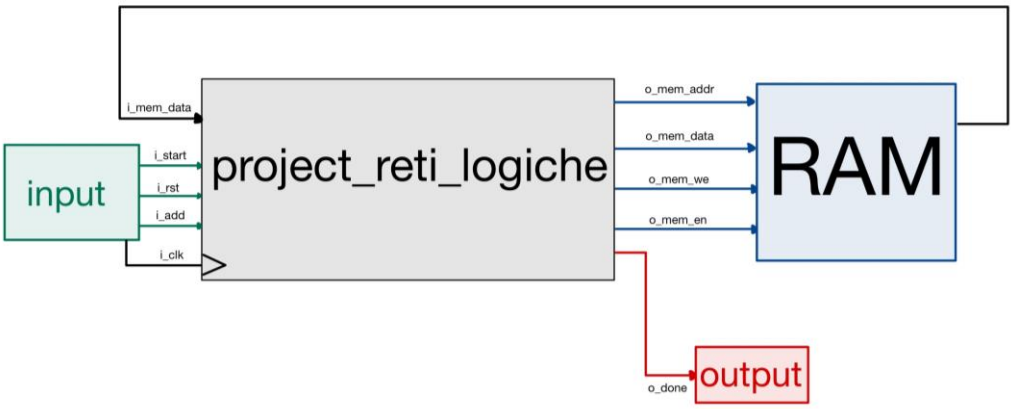
Progetto reti logiche a.a. 2025

Gruppo di Matteo Piacentini (Cod. Per. 10896104) e Lorenzo Zani (cod. per. 10847215)

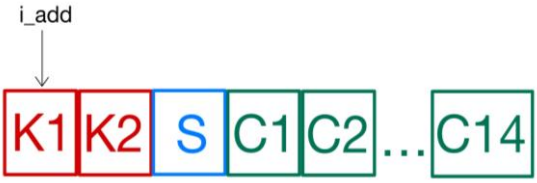
Introduzione

L'obiettivo del progetto è quello di realizzare e sintetizzare nel linguaggio VHDL un componente che implementi la specifica che viene descritta e commentata come segue.

L'interfaccia di ingressi e uscite del componente è rappresentata nello schema riportato sotto. Il componente estrae una serie di dati letti sequenzialmente da una RAM e gli applica un filtro a scorrimento, scrivendo il risultato nelle celle di memorie successive all'ultimo valore letto.



A partire dall'indirizzo in ingresso, il componente dovrà leggere un header di 17 byte, che conterrà le informazioni necessarie per svolgere i calcoli:



- K1, K2: insieme rappresentano un numero di 16 bit che è il numero di byte che il componente leggerà in successione.
- S: il cui bit meno significativo indicherà il filtro che verrà applicato ai dati: 0 per il filtro di tipo 3, altrimenti 5.
- C1, ..., C14: quattordici byte che contengono i valori dei filtri, da C1 a C7 i valori del filtro di tipo 3 e da C8 a C14 i valori del filtro di tipo 5.

Ciascun filtro è caratterizzato da 7 coefficienti, un fattore di normalizzazione e una dimensione.
Nel filtro di tipo 3 i valori in posizione 0, 3, 6 sono sempre nulli, possiede un fattore di normalizzazione pari a 12 ed una dimensione di 2+2+1=5.
Il filtro di tipo 5 non possiede valori certamente nulli, possiede un fattore di normalizzazione pari a 60 ed una dimensione di 3+3+1=7.

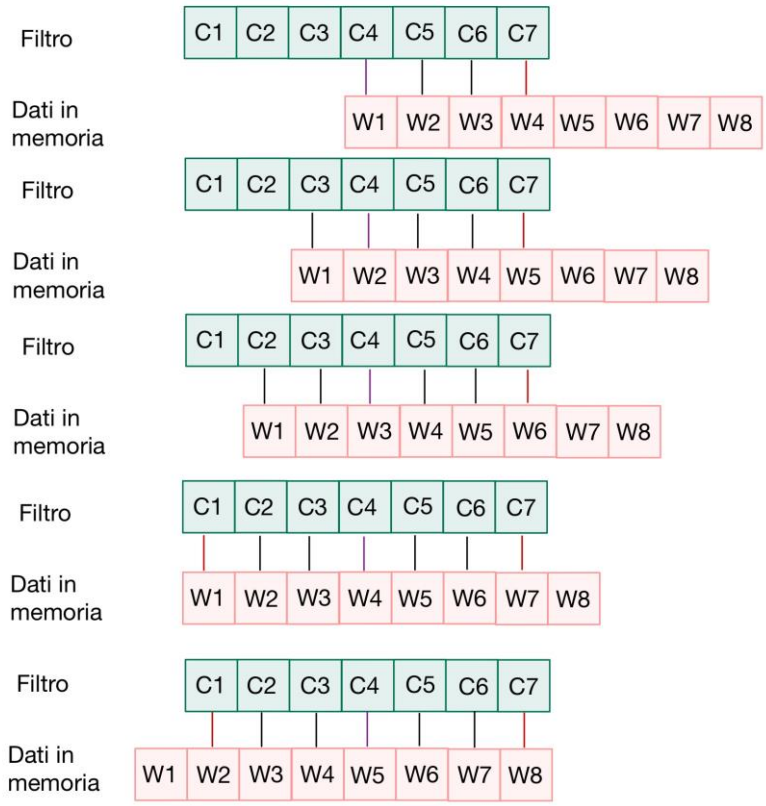
Il componente applica ai valori di ingresso la funzione $f'(i)$ definita come:

$$f'(i) = (1/n) \cdot \sum C_j \cdot f[j+i]$$

Dove:

- $n :=$ rappresenta il fattore di normalizzazione del filtro
- $i :=$ rappresenta l'i-esimo elemento preso dalla memoria (parola W_i)
- $f[j+i] :=$ rappresenta la parola W_{j+i} presa dalla memoria
- $C_j :=$ rappresenta il j-esimo valore del filtro
- $j :=$ rappresenta la variabile di iterazione della sommatoria lungo la dimensione della maschera.

Descriviamo l'attività del componente con un esempio pratico. Supponiamo di scegliere un filtro di tipo 3 e dover applicarlo su 8 byte.



Dalla figura osserviamo che il filtro scorre lungo i valori della memoria, di volta in volta incrementando di uno la posizione del centro del filtro a partire dal valore di W1.

I valori non presenti in memoria durante l'applicazione del filtro vengono sostituiti da zeri virtuali di padding, sia all'inizio sia alla fine della lettura dei dati.

I collegamenti in figura fra C1 e C7 e i dati in memoria (in rosso) rappresentano valori che non verranno presi in considerazione per calcolo di un filtro di tipo 3 poiché oltre la dimensione della maschera.

Chiariamo con un esempio pratico come venga calcolata la funzione: supponiamo di utilizzare un filtro di ordine 3 dato dai coefficienti [0, 2, 4, 0, 7, -3, 0], dunque con $n = 12$ e dimensione 5, e di presentare in ingresso le 8 parole [1, 2, 9, -3, 4, 0, -9, 6].

Dunque, otterremo:

$$f'(i) = (1/12) \cdot (C_2 \cdot f[i-2] + C_3 \cdot f(i-1) + C_4 \cdot f(i) + C_5 \cdot [i+1] + C_6 \cdot f[i+2])$$

Come osservato in precedenza, i primi 2 valori letti dalla memoria sono rimpiazzati da zeri virtuali di padding.

Alla prima iterazione (i=0) avremo: $f'(0) = (1/12) \cdot [(2 \cdot 0) + (4 \cdot 0) + (0 \cdot 1) + (7 \cdot 2) + (-3 \cdot 9)]$

Mentre la prima iterazione per cui utilizzeremo tutti i coefficienti del filtro sarà per i=2:

$$f'(2) = (1/12) \cdot [(2 \cdot 1) + (4 \cdot 2) + (0 \cdot 9) + (7 \cdot -3) + (-3 \cdot 4)]$$

Occorre ora normalizzare il risultato dividendolo per n (fattore di normalizzazione).

Considerata il costo di implementare la divisione di un intero per un divisore non potenza di due, possiamo approssimare i due divisori di normalizzazione come somme di reciproci di potenze di due, in modo da ottenere una buona stima del valore reale e una facile implementazione. Infatti, in aritmetica binaria, dividere per un valore della forma 2^n equivale a shiftare il dividendo di n bit verso destra.

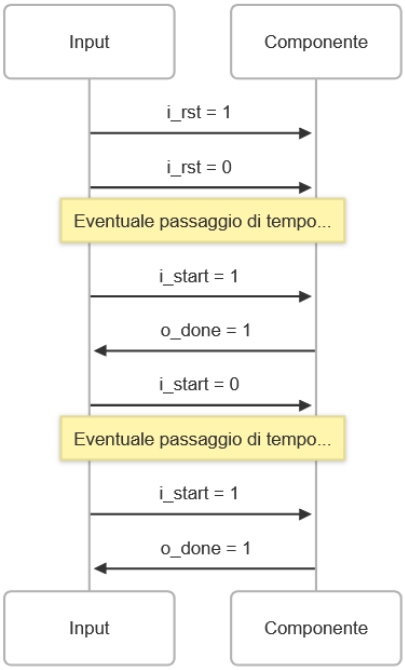
Verifichiamo con i calcoli, sapendo che:
1/16 = 0.0625
1/64 = 0.015625
1/256 = 0.00390625
1/1024 = 0.000976562
Possiamo vedere le approssimazioni

- 1/12 = 0.083333...
  1/12 approssimato a 1/16 + 1/64 + 1/256 + 1/1024 = 0.083007812
- 1/60 = 0.016666...
  1/60 approssimato a 1/64 + 1/1024 = 0.016601562

Per cui l'approssimazione descritta per la divisione è sufficientemente accurata e semplifica significativamente la sintesi.

Occorre notare però che l'operazione di shift right aritmetico in 2c2 dà origine ad una divisione per due fra interi che porta i numeri positivi verso lo zero e i numeri negativi verso il valore –1. Per via di questa asimmetria diventa necessario aggiungere 1 al risultato dello shift di valori negativi per evitare errori numerici durante la normalizzazione (ad esempio, senza questo accorgimento per il filtro di tipo 3 il valore –1 verrebbe normalizzato a –4 invece che a 0).

Dopo che l'i-esimo elemento all'indirizzo i_add + 17 + i è stato trasformato dalla funzione descritta sopra, questo deve essere scritto in memoria all'indirizzo i_add + 17 + i + K, dove K è il numero di byte di cui è composto l'input.



È di rilievo nella descrizione del componente anche il protocollo di interazione fra un eventuale utilizzatore e il componente stesso.

Da specifica, il componente può contare su una prima fase di reset nel quale il suo stato interno viene inizializzato.

Da quel momento, quando il segnale i_start viene alzato il componente raccoglie i valori in ingresso e calcola il risultato dell'applicazione del filtro, asserendo o_done quando l'intero calcolo è svolto.

Successive attivazioni del componente potrebbero avvenire senza un esplicito segnale di reset, e questo va gestito nell'implementazione.

Descriviamo ora l'architettura progettata per il componente. Si segnala che in alcune sezioni della descrizione verranno segnalate le fonti dalla quali vengono estratte alcune informazioni, in particolare nel commento della sintesi. Le fonti impiegate sono comunque riportate nella bibliografia in fondo alla relazione.

Descrizione dell'architettura

L'architettura nel suo insieme consiste nel controllo di tre sottocomponenti strutturali che vengono instanziati all'interno di un unico componente principale (project_reti_logiche) e nel controllo degli accessi alla memoria.

Ciascun singolo componente realizza una funzione logica indipendente dalle altre, e il componente principale agisce sui loro segnali di controllo per permettere l'ingresso dei dati letti dalla RAM nel ciclo del calcolo del filtro a scorrimento.

In particolare, i tre sotto componenti di cui verrà data una descrizione dettagliata singolarmente sono:

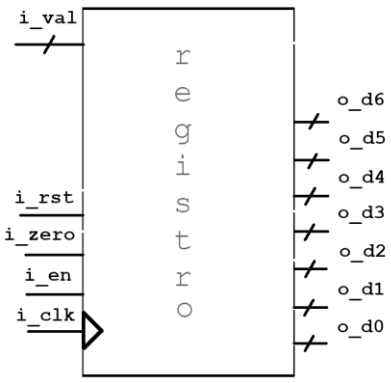
i) **shift_register**: Un registro a scorrimento di 7 byte utilizzato per immagazzinare i dati letti di volta in volta dalla RAM e sfruttare le precedenti letture, in modo da necessitare (dopo la lettura dei primi 4 valori) una singola lettura per ogni nuovo valore calcolato. Quando abilitato, ad ogni ciclo di clock i byte scorrono di una posizione e lasciano posto ad un nuovo byte estratto dalla memoria.

ii) **kernel**: Un semplice componente che realizza il prodotto fra i fattori selezionati dalla ram e i coefficienti (quindi il filtro). La ratio dietro l'isolamento di questo calcolo sta nell'ispezione del design sintetizzato dall'elaboratore di VHDL per verificare che venisse correttamente implementata una struttura "ad albero" per minimizzare la latenza.

iii) **out_normalizer:** Componente che realizza il calcolo della divisione approssimata (diversa per i due tipi di filtro) e implementa sia la correzione descritta precedentemente prevista per i numeri negativi, sia la limitazione dei valori in uscita nell'intervallo [-128, 127].

I tre sottocomponenti sono controllati da project_reti_logiche, che asserisce su necessità i segnali di controllo del registro a scorrimento e quelli atti alla lettura della ram, in modo da coordinare tutti e tre sui fronti d'onda del clock. Una descrizione dettagliata dell'intreccio dei vari componenti ad opera di project_reti_logiche verrà fornita dopo la descrizione singola dei tre, onde prevenire ambiguità.

I. Registro a scorrimento (shift_register)



Il modulo shift_register rappresenta un registro di 7 byte con scorrimento sincrono al fronte di salita del clock di riferimento.

La sua implementazione è un semplice blocco con un singolo processo sensibile ai segnali di controllo del registro, che sono i_en, i_zero e i_rst, oltre che al clock. In quel processo avviene lo shift dei byte sul fronte di ascesa del clock e viene gestita la flag i_zero.

L'attività dei segnali di controllo è la seguente:

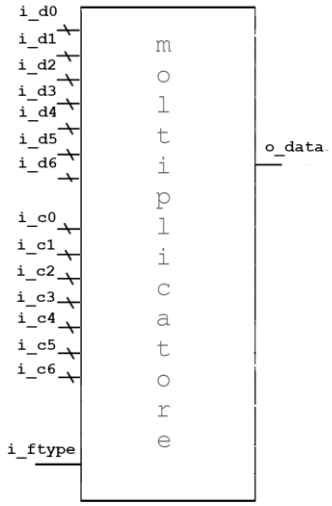
- i_en: Segnale di enable, attiva o disattiva lo scorrimento del registro. Quando attivo, sul fronte d'onda di salita del clock il registro scorre di un byte.
- i_zero: Quando asserito, se il registro è attivo viene scritto il valore 0x00 nella prima posizione scorrevole del registro durante il fronte di salita del clock.
- i_rst: Quando attivo e in maniera asincrona resetta lo stato di tutti i byte nel registro a scorrimento.

Quando il segnale i_zero non è asserito e i_en è alto, il valore contenuto nell'ingresso i_val viene scritto nella prima posizione scorrevole del registro.

Poter controllare quali valori vengono introdotti nel registro (valori in ingresso dalla ram o valori nulli per via della flag i_zero) permette di gestire il caso limite del calcolo della maschera per gli ultimi valori in memoria, per il quale il filtro viene convoluto contro degli "zero virtuali" di padding.

I 7 byte del registro sono riportati in uscita e fungono da ingresso al componente moltiplicatore, come verrà descritto in seguito.

II. Moltiplicatore (kernel)



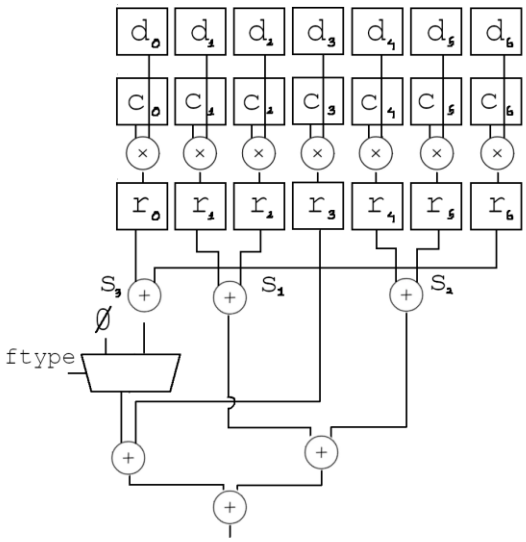
Il modulo moltiplicatore rappresenta l'unità di calcolo principale del componente e prende in ingresso i 7 byte da moltiplicare assieme ai 7 coefficienti, salvati internamente nel componente principale dopo l'inizializzazione, con i quali effettuare il calcolo della maschera, oltre che il tipo di filtro che il componente deve implementare.

L'implementazione stessa del blocco è puramente combinatoria (senza quindi alcun processo) e si limita a moltiplicare i segnali di ingresso e riportare in uscita la somma complessiva.

A seconda del tipo di filtro, alcune posizioni potrebbero essere azzerate, come nel caso delle posizioni più esterne nel filtro di tipo 3.

Fra tutti i componenti, questo blocco rappresenta quello con il delay di propagazione più elevato per la natura intensa dei calcoli (implementazione della moltiplicazione fra più valori e numerose somme intermedie).

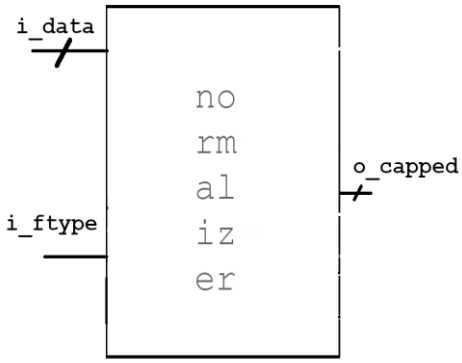
Per questo motivo la struttura scelta per implementare il calcolo del filtro è stata progettata appositamente a forma di albero, in modo da minimizzare la latenza per il calcolo del filtro (se le somme fossero sequenziali il numero di livelli nel circuito sarebbe raddoppiato).



A tal fine l'implementazione del blocco presenta numerosi segnali intermedi che istruiscono lo strumento di design RTL di Vivado nel costruire una struttura ad albero.

Nella seguente analisi post sintesi verrà confermato che Vivado ha sintetizzato la struttura ricercata per il moltiplicatore.

III. Normalizzatore (normalizer)



Il modulo normalizzatore rappresenta l'unità che raccoglie il risultato del moltiplicatore e applica la normalizzazione descritta nell'introduzione, inclusa la correzione per i numeri negativi e il cap all'intervallo [-128, 127] per l'uscita.

L'implementazione stessa del modulo è un semplice blocco combinatorio che raccoglie l'ingresso e il tipo di filtro (indicato come un segnale logico binario) e calcola la giusta normalizzazione.

Due valori sono calcolati separatemente: il termine in comune nella normalizzazione, ovvero

T_0 = (1/64 + 1/1024)*i_data

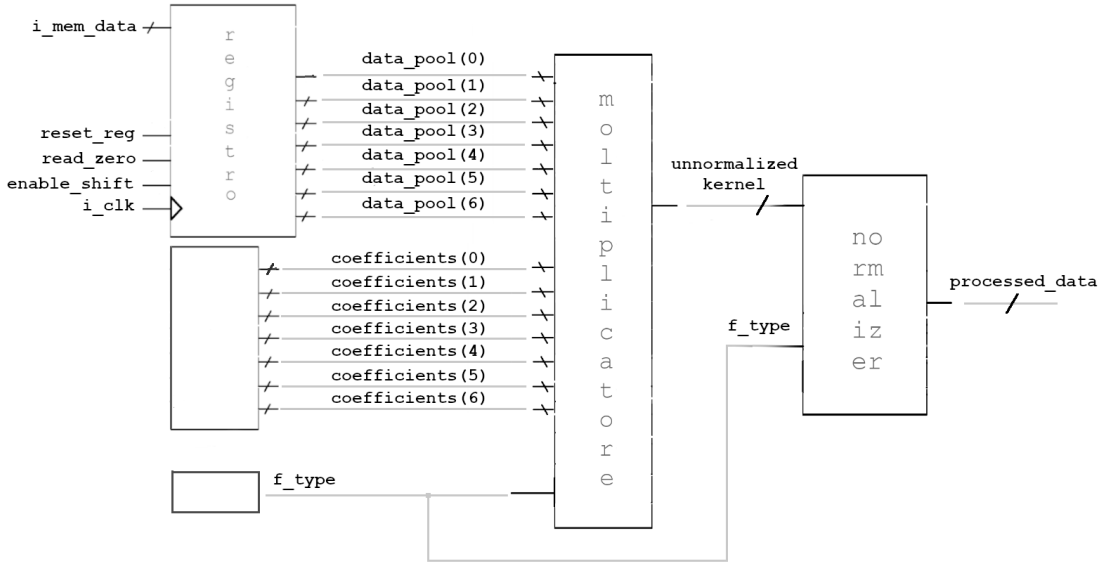
E il termine specifico della normalizzazione per il filtro di livello 3

T_1 = (1/16 + 1/256)*i_data

Tutte le divisioni sono naturalmente implementate come shift aritmetici (quindi con estensione di segno) verso destra tramite la funzione nativa shift_right di VHDL. Gli operatori nativi srl e sll di vhdl sono stati evitati per la loro specifica ambigua, come riportato nella fonte [7] presente nella bibliografia alla fine della relazione.

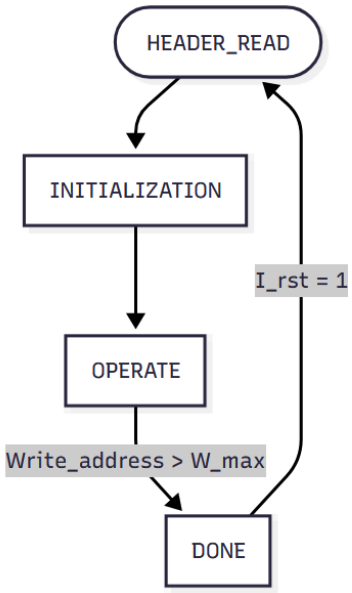
Tramite un selettore, viene riportato in uscita o solo il termine comune o la somma dei due termini in base al tipo di filtro, al quale viene aggiunto un fattore di correzione in base al segno del valore in ingresso in modo da evitare il problema descritto nell'introduzione per la normalizzazione di numeri negativi.

IV. Descrizione dettagliata del componente principale



Viene fornita ora una descrizione del componente primario nella sua interezza, il cui merito principale è instanziare gli altri tre componenti e gestirne i segnali di controllo, oltre che comunicare con il banco di memoria.

I valori dei segnali coefficients(0...6), data_pool(0..6), f_type sono elementi di memoria interni al componente e vengono inizializzati prima dell'uso nel calcolo del filtro come descritto nel seguito.



Il componente principale nel suo insieme rappresenta una macchina con 4 macro-stati le transizioni tra i quali dipendono dall'ingresso del componente e dalle attività interne della macchina.

L'implementazione interna è un singolo processo sensible ai segnali di controllo i_rst (asincrono) e i_clk che implementa la macchina a stati e il controllo dei segnali di memoria e controllo dei componenti.

Ciascun macro-stato è raffinato in ulteriori stati che vengono descritti di seguito.

I quattro macro stati sono:

1. HEADER_READ: Il componente sta leggendo l'header salvandone i dati rilevanti. Nell'implementazione vengono compilati i segnali intermedi rappresentanti i coefficienti, il tipo di filtro e il numero di valori da esaminare nella memoria, corrispondenti ai segnali interni f_type, coefficients(0...6) e data_amt. Completato questo stadio, la macchina passa ad INITIALIZATION.

2. INITIALIZATION: Il componente sta cominciando a leggere la memoria, ma non è ancora pronto ad operare. Bisogna infatti caricare i primi 3 valori prima di iniziare ad applicare la maschera, come descritto nell'introduzione. Nell'implementazione, in questa fase il componente istruisce il registro a scorrimento di caricare i valori letti dalla memoria. Completato questo stadio, la macchina passa ad OPERATE.

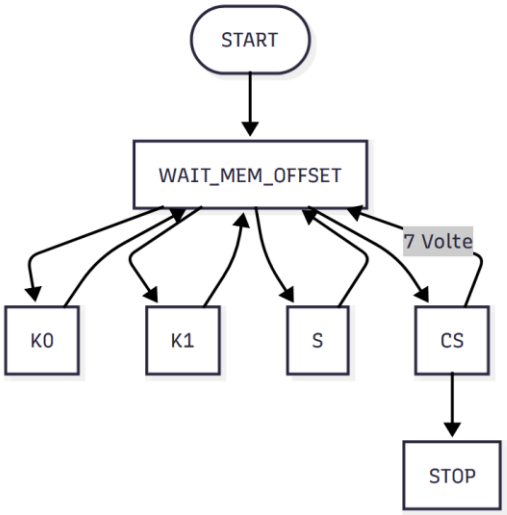
3. OPERATE: Il componente sta calcolando i valori previsti dalla funzione e li sta scrivendo nell'area di memoria designata. In questa fase viene abilitato di volta in volta il segnale di scrittura o_mem_we. Una descrizione dettagliata dell'operazioni della macchina in questo stato viene fornita in seguito. Completato questo stadio, la macchina termina nello stato di DONE.

4. DONE: Il componente ha esaurito i calcoli da eseguire ed è in attesa di reset o di un nuovo segnale di start da parte dell'utilizzatore. In entrambi i casi, la macchina viene portata nello stadio di HEADER_READ e ricomincia il calcolo dei valori.

Ciascun macro-stato è suddiviso in più fasi che comportano un'attività diversa del componente. Viene quindi ora descritta in dettaglio l'architettura di ciascuno stato della macchina.

Si vuole evidenziare che per non appesantire il codice e la rappresentazione dei FSA, gli stati di attesa per la propagazione dei segnali e della memoria, descritti nel seguito, sono spesso unificati in un unico stato, e l'informazione sullo stato futuro viene immagazzinata temporaneamente in una variabile ausiliare after_waiting, sicché formalmente lo stato reale della macchina sarebbe dato dalla coppia (current_state, after_waiting).

1. Macro-stato di HEADER_READ



Nella fase di HEADER_READ, il componente legge l'header all'interno di variabili interne che rappresentano il numero di valori da leggere (K0 concatenato a K1), il tipo di filtro S e i sette coefficienti CS.

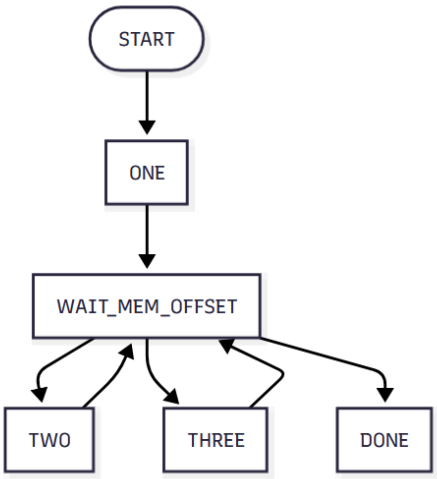
Ad ogni lettura, il prossimo stato nella macchina viene salvato in una variabile ausiliaria e la macchina viene portata in un sotto stato di attesa (WAIT_MEM_OFFSET) nel quale il componente è inattivo e attende che la lettura venga completata dalla RAM.

Brevemente, gli stati interni sono:

1. START: Vengono inizializzate variabili ausiliarie per l'indirizzo di lettura e l'indice di lettura.
2. WAIT_MEM_OFFSET: Stato di attesa e inattività. Al prossimo ciclo di clock viene ripristinato lo stato prossimo "reale".
3. K0: Viene letto il valore K0 e immagazzinato nel byte superiore di un unsigned di 16 bit.
4. K1: Viene letto il valore K1 e immagazzinato nel byte inferiore dell'unsigned di 16 bit.
5. S: Viene letto il tipo di filtro dall'ultimo bit del byte letto. A seconda del tipo di filtro, l'indice di lettura può saltare i 7 coefficienti di tipo 3.
6. CS: Per 7 volte, il valore dei coefficienti viene salvato internamente in segnali mappati sugli ingressi del moltiplicatore.
7. STOP: Dopo aver letto tutti i valori, la macchina segna nel prossimo macro-stato il valore INITIALIZATION e resetta il registro a scorrimento. A partire dal prossimo ciclo di clock, esso verrà inizializzato con i primi valori.

   Quando tutti i valori sono letti, la macchina viene portata nello stato di inizializzazione del registro a scorrimento.

2. Macro-stato di INITIALIZATION

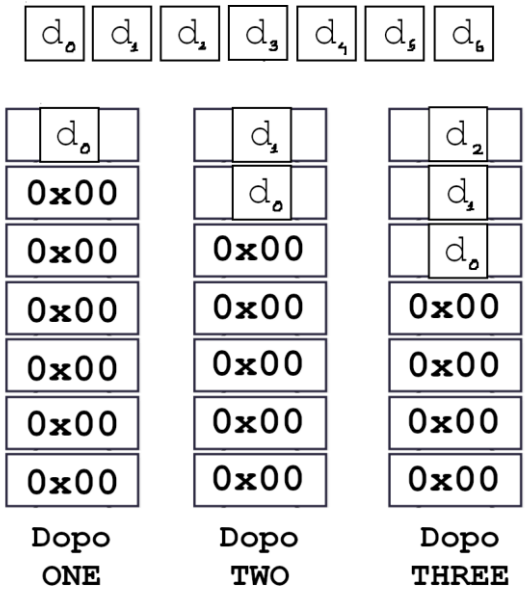


Nella fase di INITIALIZATION, il contenuto del registro a scorrimento viene inizializzato con i primi tre valori dell'area di memoria indicata. Il quarto valore verrà letto nella prima fase del prossimo macro-stato.

Si fa notare che in base alla dimensione del filtro il numero dei valori richiesti per calcolare il primo valore varia da 3 a 4, ma ne viene letto un numero fisso in questa fase: questo dettaglio è gestito congiuntamente dal modulo moltiplicatore (che in base al valore del segnale di ingresso f_type esclude i valori più estremi della maschera dal calcolo) e dal controllo del segnale i_zero del registro a scorrimento nella fase di OPERATE successivamente descritta.

Brevemente, gli stati sono:

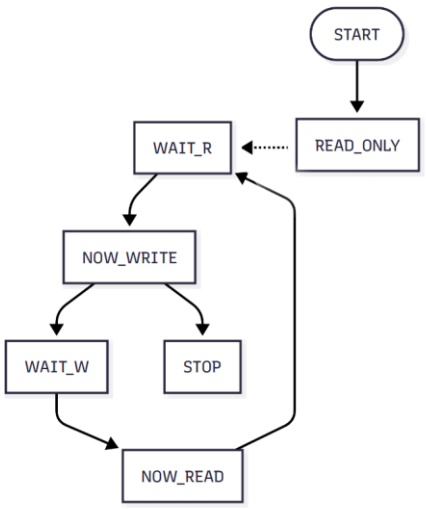
1. START: Viene portato a zero il segnale di reset del registro, che era stato attivato nell'ultimo sotto-stato di HEADER_READ. Inizia quindi la lettura della memoria.
2. ONE, TWO, THREE: I primi tre valori della memoria vengono letti e immagazzinati nel registro a scorrimento. Di volta in volta, i valori sono presentati sulla porta di ingresso del registro e viene abilitato lo scorrimento del registro tramite l'appropriato segnale di controllo.
3. DONE: Il macro-stato della macchina viene portato ad OPERATE.



Viene mostrato nella figura il contenuto del registro a scorrimento dopo l'attività di INITIALIZATION.

Prima di portare lo stato ad OPERATE, il segnale di controllo dell'enable shift per il registro viene portato a zero in attesa della lettura del d_3 che seguirà nei cicli di clock successivi, nelle prime fase di OPERATE.

3. Macro-stato di OPERATE



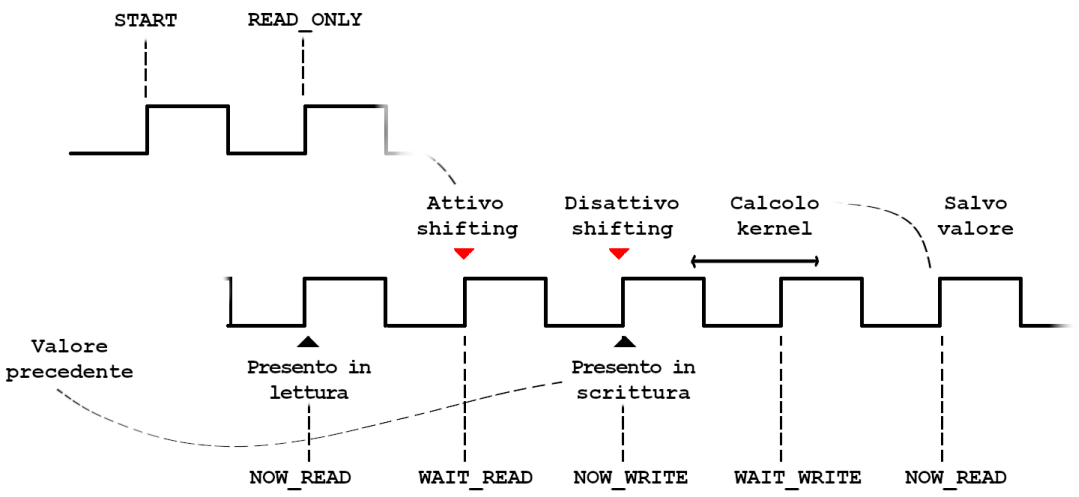
Nel macro-stato di OPERATE vengono calcolati i valori del filtro e vengono scritti sulla memoria uno ad uno.

Ad ogni passo della macchina viene letto un nuovo valore nel registro a scorrimento e mentre viene calcolato il risultato del filtro viene scritto sulla memoria il risultato del precedente calcolo. Questa operazione si distribuisce su 4 stati che corrispondono a 4 cicli di clock consecutivi, oltre che a due stati di START e READ_ONLY che descrivono l'ingresso della macchina a metà ciclo.

In questa fase devono essere coperti alcuni corner cases derivanti dall'attività di calcolo:

1. La fine dei valori in ingresso per gli ultimi valori della maschera nel qual caso vanno riportati byte nulli al registro a scorrimento.
2. Nel primo ciclo di operazione nessun valore dev'essere scritto in quanto non calcolato in precedenza.
3. Il verificarsi della condizione che porta allo STOP della macchina, ovvero la scrittura dell'ultimo valore.

Di sotto è riportato uno schema che mostra gli eventi principali del ciclo che è d'ausilio per la descrizione dei sotto-stati di questo processo.



Nel dettaglio i sottostati sono:

1. START: Viene disabilitato l'enable per il registro a scorrimento e vengono inizializzati i valori ausiliari di max_read_address e max_write_address, che saranno usati per gestire i casi 1 e 3 descritti precedentemente.

2. READ_ONLY: Si procede alla lettura di un singolo valore. Questo stato differisce da NOW_READ per il fatto che non comprende l'assegnamento del valore calcolato precedentemente (che non è ancora stato calcolato a questo punto).

3. NOW_READ: Si procede non solo alla lettura del singolo valore in ingresso ma anche a salvare il valore calcolato in precedenza durante l'attesa per una scrittura. Viene verificato se l'indirizzo da leggere è minore di max_read_address, se così non fosse viene attivato il segnale di controllo read_zero mappato sul segnale in ingresso i_zero del registro a scorrimento, per gestire il caso 1.

4. WAIT_READ: Stato di attesa per il completamento della lettura. Viene abilitato lo shift del registro a scorrimento, che scorrerà per fare spazio al nuovo valore letto.

5. NOW_WRITE: Si procede a scrivere in memoria il valore precedentemente calcolato, se questo è stato effettivamente calcolato (gestendo quindi il caso limite 2). Inoltre viene verificato se l'indirizzo di scrittura è minore di max_write_address, e in caso contrario l'operazione della macchina prosegue nell'ultimo sotto-stato di STOP, gestendo la conclusione del processo (il caso limite 3).

6. WAIT_WRITE: Stato di attesa per il completameto della scrittura e per il delay di propagazione attraverso il moltiplicatore dei dati.

7. STOP: Viene alzato il valore di o_done e la macchina entra nello stato DONE.

L'implementazione della FSA descritta si riduce ad uno switch sui possibili valori degli stati e alla manipolazione dei segnali di controllo per la memoria e il registro a scorrimento all'interno di un processo.

4. Macro-stato di DONE

Nello stato di done la macchina è in uno stato complessivo di attesa per un nuovo segnale di start o per un segnale di reset.

Quando il segnale di start viene posto a zero o il segnale di i_rst è asserito la macchina viene ripristinata allo stato iniziale ed è possibile effettuare nuovi calcoli con nuove combinazioni di valori in ingresso.

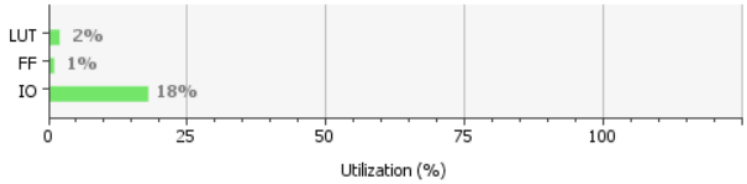
Questo stato è completamente inattivo e la sua implementazione si riduce all'azione del processo di reset.

Risultati sperimentali – Sintesi

Nella stesura di questi commenti sono state utilizzate varie fonti che vengono riportate nella bibliografia che segue l'analisi. Parte delle informazioni sono state estratte dal forum Adaptive SoC & FPGA Support di AMD, mentre il restante deriva da forum online dedicati allo sviluppo di FPGA o dalla documentazione di Vivado.

A. Utilization report

Resource	Utilization	Available	Utilization %
LUT	877	41000	2.14
FF	252	82000	0.31
IO	54	300	18.00



Il componente ha un utilizzo delle LUT [2] molto ridotto, appena il 2.14%. Ciò è atteso in quanto la funzione calcolata è piuttosto semplice e il componente è una semplice macchina a stati.

I flip flop (FF) utilizzati sono in numero ancora più ridotto, appena lo 0.31%. Difatti il componente progettato non impiega significativi buffer di memoria oltre a salvare internamente l'header e gli ultimi sette valori letti dalla memoria.
Se fosse noto da specifica che la RAM presenta agevolazioni per il tempo richiesto da lettura sequenziali, volendo minimizzare il tempo di operazione sarebbe stato invece vantaggioso mantenere un buffer interno al componente.

L'utilizzo delle porte IO[3] è pari a 54/ 300 come atteso dalle specifiche, in quanto si hanno 27 bit per i segnali di ingresso e 27 bit per i segnali di output.

Analizziamo adesso i singoli componenti.

Name	Slice LUTs (41000)	Slice Registers (82000)	Bonded IOB (300)	BUFGCTRL (32)
project_reti_logiche	877	252	54	1
KERNEL (convolutor)	217	0	0	0
OUT_NORMALIZER (normali...	32	0	0	0
SH_REGISTER (shift_register)	340	56	0	0

In project_reti_logiche osserviamo la presenza di un componente BUFGCTRL [4], che riguarda la gestione del clock in ingresso.

Osserviamo che i due componenti puramente combinatori KERNEL e OUT_NORMALIZER non utilizzano slice registers. Lo shift register invece possiede sufficienti registri per garantire il buffer interno di 7*8 = 56 bit.

Infine riportiamo di seguito un elenco di ciò che è stato usato da Vivado per sintetizzare il componente scritto:

Ref Name	Used	Functional Category
LUT2	328	LUT
LUT6	253	LUT
LUT4	249	LUT
FDRE	169	Flop & Latch
CARRY4	165	CarryLogic
LUT5	127	LUT
LUT3	118	LUT
FDCE	76	Flop & Latch
LUT1	60	LUT
OBUF	27	IO
IBUF	27	IO
FDSE	7	Flop & Latch
BUFG	1	Clock

B. Timing Report

Il seguente Timing report è stato eseguito dopo aver creato un file .xdc contenente la seguente riga di codice, che imposta il constraint indotto dal clock al suo minimo valore di 20 ns.

```
create_clock -period 20.000 -name sys_clk_20ns -waveform
{0.000 10.000} [get_ports i_clk]
```

Riportiamo il timing report nella sua interezza nella figura sottostante.

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	8,943 ns	Worst Hold Slack (WHS):	0,068 ns	Worst Pulse Width Slack (WPWS):	9,650 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	473	Total Number of Endpoints:	473	Total Number of Endpoints:	253

All user specified timing constraints are met.

1) Setup:

Il Worst Negative Slack [1] (WNS) è ampiamente positivo (8,943ns). Avendo progettato il componente per attendere un intero ciclo di clock durante le letture e con le accortezze per la struttura ad albero del moltiplicatore, questo risultato è positivo ed atteso.

Il Total Negative Slack (TNS) è pari a 0 ns, segno che tutti i segnali vengono temporizzati correttamente nel design sintetizzato.

2) Hold

Il Worst Hold Slack (WHS)[1, 5] è pari a 0,068 ns, questo rappresenta che tutti i percorsi di hold sono rispettati, e che i dati impiegano 0,068 ns per essere salvati correttamente.

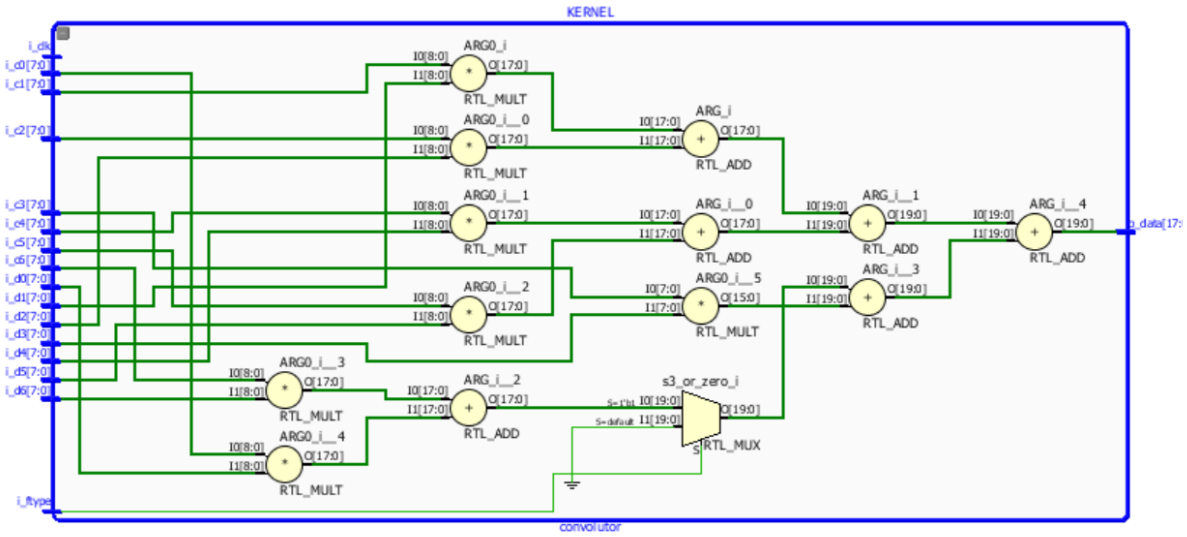
Altri indicatori, come il Total Hold Slack (THS) e il Worst Pulse Width Slack[6] (WPWS) non segnalano problemi nella temporizzazione.

Timing Check	Count	Worst Severity
no_clock	0	
constant_clock	0	
pulse_width_clock	0	
unconstrained_internal_endpoints	0	
no_input_delay	26	High
no_output_delay	27	High
multiple_clock	0	
generated_clocks	0	
loops	0	
partial_input_delay	0	
partial_output_delay	0	
latch_loops	0	

Infine, esaminando i warning di Vivado osserviamo che sono segnalati solamente i pin di uscita e ingresso che non presentano un delay esplicitamente specificato (come assunto dalla specifica). Nessun altro warning è presente.

C. Osservazioni della sintesi

Riportiamo nella figura sottostante l'elaborazione RTL per il modulo moltiplicatore.



Come commentato in precedenza, il design sintetizzato da Vivado per il moltiplicatore rispecchia la struttura ad albero desiderata e minimizza quindi il tempo di latenza per il calcolo.

Risultato dei test

Oltre ai testbench forniti dal docente, abbiamo integrato ai test di altri studenti condivisi sul forum di WeBeep con alcuni test costruiti su misura per valutare la reazione del componente su vari casi limite di uso.

Si riporta che per ogni test prodotto autonomamente sono state create due versioni, una per il filtro di tipo tre e una per il filtro di tipo cinque, per verificare che venissero coperte tutte le condizioni di operazione del componente.

Inoltre, tutti i test descritti sono stati utilizzati anche come test post-sintesi funzionali e temporali senza problemi.

Descriviamo ora sinteticamente ciascun test e il suo scopo:

I. Nome del test: "Max_data_amt" (per filtro tre e cinque)
Questo test valuta se il componente è in grado di accedere correttamente a tutta la memoria nel caso l'input indicasse di calcolare il filtro per il numero massimo di valori possibili (corrispondenti a circa metà RAM, eccetto i diciassette byte per l'header).
Il componente supera questo test anche grazie all'uso della flag i_zero per il registro a scorrimento e il controllo sull'indirizzo di scrittura nella fase di OPERATE.

II. Nome del test: "Min_data_amt" (per filtro tre e cinque)
Dualmente a max_data, questo test prende in considerazione il numero minore possibile di dati in ingresso (dalla specifica questo valore è di 7 byte)
Come per "Max_data_amt", la presenza del controllo per l'attivazione della flag i_zero per il registro a scorrimento nella fase di lettura in OPERATE è in grado di gestire questo caso limite e il componente supera il test.

III. Nome del test: "Full_of_zero" (per il filtro tre e cinque)
Questo test valuta il caso limite descritto dalla specifica per la normalizzazione dei valori negativi, che richiede di sommare 1 al valore shiftato.
Il risultato atteso è composto solamente da zero, per cui viene testato se il normalizzatore è in grado di mantenere nullo il valore zero (altro caso limite) e se è in grado di annullare piccoli valori negativi come richiesto.

IV. Nome del test: "Overflow" (per il filtro tre e cinque)
Verifica che venga correttamente calcolato il valore più grande possibile nel modulo moltiplicatore (pari a –128*-128*7) senza subire un overflow e che questo venga normalizzato correttamente, venendo quindi limitato a 127.

V. Nome del test: "Underflow" (per il filtro tre e cinque)
Verifica che venga correttamente calcolato il valore più piccolo possibile nel modulo moltiplicatore (pari a –128*127*7) senza subire un overflow e che questo venga

normalizzato correttamente, venendo quindi limitato a 127.

VI. Nome del test: “Reset_multiple_times” (per il filtro tre e il filtro cinque)
Questo test verifica che il componente sia in grado di resettarsi correttamente più volte.
Il componente supera questo test grazie alla procedura asincrona di reset implementata, che porta al valore iniziale tutte le variabili e i segnali coinvolti nel calcolo.
Inoltre, il reset è testato anche quando l’esecuzione del componente è già avviata da numerosi cicli di clock. Anche questo aspetto viene gestito correttamente dal componente.

VII. Nome del test: “Restart_with_no_reset” (per il filtro tre e cinque)
Questo test verifica come da specifica che il componente sia in grado di eseguire più calcoli consecutivi senza necessitare di un segnale di reset dopo la prima attivazione del componente.
Nell’implementazione questo viene gestito dalla stessa procedura del reset.
In aggiunta, questo test verifica che o_done venga gestito come da specifica dal componente, ovvero venga asserito solo dopo la fine del componente e che venga abbassato quando in seguito i_start viene abbassato.

VIII. Suite di test generati con il tool online[8] condiviso sul forum
Questa suite di test è stata impiegata per generare la correttezza generale dei calcoli eseguiti dal componente, partendo da indirizzi di memoria casuali e calcolando filtri con coefficienti casuali, per verificare congiuntamente che il componente potesse accedere a tutta la memoria correttamente senza problemi e che l’implementazione della funzione calcolata fosse corretta.

Conclusioni

Partendo da un documento di specifica, abbiamo realizzato e sintetizzato un componente che realizza il calcolo di un filtro a scorrimento interfacciandosi con un banco di memoria ad accesso randomico.

Nonostante le numerose difficoltà emerse dovute alla necessità di mettere in pratica in autonomia quanto appreso a lezione in merito a VHDL, dopo una fase di sviluppo e correzione iterativa del codice (supportata dalla scrittura contemporanea di alcuni test), il componente realizzato supera i test a cui viene sottoposto, sia pre- che post-sintesi, e supera il requisito di clock minimo.

Viene anche confermato dai report di sintesi di Vivado che internamente al componente i segnali vengono temporizzati correttamente e sono propagati in tempo, così che il componente risulta correttamente implementabile.

Inoltre, un'ispezione del design mostra che i sotto-componenti di moltiplicazione e normalizzazione rispecchiano il design strutturale preliminare, e come atteso l'implementazione in net-list occupa uno spazio ridotto e un numero di componenti di memoria limitato al buffer interno.

Complessivamente, il progetto è stato per entrambi una buona opportunità di mettere in pratica quanto appreso su VHDL.

Bibliografia

1. https://adaptivesupport.amd.com/s/question/0D54U00006qTJxwSAG/what-is-the-worst-negative-slack?language=en_US
2. https://electronics.stackexchange.com/questions/169532/what-is-an-lut-in-fpga
3. https://docs.amd.com/r/en-US/ug906-vivado-design-analysis/Global-Clocks-Tables
4. https://www.fpgadeveloper.com/2011/09/code-templates-clock-mux.html/
5. https://electronics.stackexchange.com/questions/208774/what-is-the-difference-between-worst-hold-slack-and-worst-negative-slack
6. https://adaptivesupport.amd.com/s/question/0D52E00006iHrRKSA0/what-is-the-pulse-width-slack?language=en_US
7. https://jdebp.uk/FGA/bit-shifts-in-vhdl.html
8. https://www.bia3ia.com/testbench-generator/